

Identity & Authorization Design — Collaborate

Author: Ciro Andrés Cobos Sánchez · **Date:** 2026-08-24

Assumptions

- 1. An existing authorization path already serves these three use cases. It is functional but does not meet the requirements in this brief; the work is a replacement under live traffic, not a greenfield build.
- 2. One email address maps to exactly one identity in one population. A person who needs access both as firm staff and as an external guest uses two addresses — account linking is therefore out of scope for the authorization layer.

S1: High-Level Architecture

Components

Component	Responsibility	Outside of scope
Database	Source of truth for workspace roles, resource overrides and firm policy	Performs no evaluation; stores only
Redis	Holds each user's resolved privilege tree for the read path	Never authoritative; a fast copy that can be rebuilt
Message bus	Carries permission changes from the write path to Sync-API, fed by an outbox	Not shared with other services; carries no decisions
Sync-API (PDP + PIP)	Resolves the fine-grained decision, owns the cache, applies invalidation events, recomputes from the database on a miss	Not a fallback Auth-API; issues no tokens
Auth-API (PAP)	Authenticates, federates to firm IdPs, exchanges tokens, writes policy, publishes changes	Never reads or writes the cache

PEP: the downstream resource endpoints (Document Service, Financial Data API, Comments Service) enforce the decision. They validate the token's coarse scopes locally and call Sync-API for the fine-grained check. They never read the permissions database.

Key Decisions

- 1. Tokens carry identity and coarse scopes, never fine-grained permissions. Coarse scopes say which API a caller may reach and change rarely. Fine-grained permissions change faster than any token TTL — embedded, they would survive until expiry, which breaks the seconds-level revocation requirement.
- 2. On a cache miss with the database unreachable, the system fails closed. Users whose privilege tree is already cached keep working; users who are not cached are denied rather than granted access we cannot verify.
- 3. The read path (Sync-API) is separated from the write path (Auth-API). Reads scale to tens of thousands per second; writes are rare. Coupling them forces one scaling profile on both, and a write-path outage would take reads down with it.
- 4. Cache invalidation is pushed onto a bus, not driven by TTL, since small TTL will transfer load to DB.
- 5. Every decision records which of the three permission planes produced it. In an audit product a denial nobody can explain is indistinguishable from a bug, and the explanation has to come from the decision itself, not from a reconstruction after the fact.
- 6. On-behalf-of is delegation, not impersonation. The exchanged token is narrowed to what the delegating user can do, and the act claim names the calling service: downstream authorizes against sub, and attributes against act.

S2: Implementation Plan

Phases are ordered by the risk each one retires, not by the components they touch.

Phase	Ships	Risk it retires	Exposure	Advance criterion
1	Decision resolver over the database: three permission planes, no cache	An explicit resource-level deny is masked by an inherited workspace allow	None	Every combination in the precedence matrix resolves as specified; explicit deny wins in all of them
2	Shadow evaluation against the path serving traffic today	The new decision disagrees with the one users get today	Read-only, no user effect	Zero divergence on denies. Allow-side divergence explained case by case, never averaged
3	Outbox on the write path, bus, cache invalidation consumer	A permission change is lost between the database and the read path	Still shadow	A revocation is visible at the decision endpoint inside the revocation-lag target

Phase	Ships	Risk it retires	Exposure	Advance criterion
4	Redis on the read path, single-flight on miss	A cold start stampedes the database; the privilege tree does not fit at real tenant size	Still shadow	Hit ratio above target on the largest tenant's tree; a cold restart does not move database p99
5	Per-firm client configuration and one federated firm IdP	A firm's IdP authenticates a user belonging to another firm	Flag, opt-in firms	An assertion issued for firm B is rejected at firm A's endpoint
6	Token exchange with narrowing and the act claim	A downstream call is not attributable to the delegating user	Flag, internal callers	sub and act reach the decision log on every hop; no output token is wider than its input
7	Canary, cutover, legacy decommission	Real traffic behaves unlike shadow traffic; two authorization paths drift apart	5% → 100% → legacy off	Denies match the legacy path across the canary window; the legacy path carries no traffic for one week before removal

Deferred: onboarding firms beyond the first federated IdP — the risk is retired once one firm works end to end, and the rest is configuration rather than engineering. Self-service editing of per-firm client configuration is also deferred: it changes who is allowed to change authorization, which needs a review of its own.

S3: Testing Strategy

Requirement	Failure it would represent	How we detect it
Login — per-firm federation	A firm's federated IdP authenticates a user who belongs to a different firm	Integration test over WebApplicationFactory, one stubbed IdP per firm: an assertion issued for firm B is rejected at firm A's endpoint
Login — two user populations	A user is routed to the wrong authentication path: an invited external user is sent to a firm's federated IdP, or firm staff is handled as a guest	Integration test over WebApplicationFactory with stubbed IdPs, data-driven over email domain → expected authentication path
Permission checking — three permission planes	An explicit resource-level deny is masked by an inherited workspace allow	Unit test over the precedence resolver — TUnit, table-driven across every (firm policy × workspace role × resource override) combination; explicit deny must win in all of them
Permission checking — revocation on long-lived sessions	Access survives past the target after a role is removed, or an open session has to re-authenticate before it loses access	Integration test over TestContainers (database, Redis, bus): revoke while a session is open, assert the next check denies. Stress test over NBomber: the same revocation at the target check rate, measuring time to first deny against the p99 target
On-behalf-of — confused deputy	Downstream authorizes against the calling service instead of the delegating user	Integration test over WebApplicationFactory: an actor token carrying wider rights than sub; assert the call is denied and that the decision followed sub
On-behalf-of — attribution for audit	The act claim is dropped on a hop and the audit trail loses the actor	Contract test at each downstream boundary: sub and act must both survive the exchange and reach the decision log

Deferred: PKCE and signature validation come from the framework and the central IdP — we assert configuration, not the protocol; re-testing them buys coverage, not confidence. Cache-to-database reconciliation is exercised by the reconciliation job's own suite, not by the authorization strategy.

S4: Evaluation & Observability

Question we need answered	Answered by	Signal	Target or trigger
Is the decision path fast enough to sit inside the request?	Metric	Decision latency at Sync-API, excluding network	p99 < 10 ms
Did a revocation actually reach the read path in time?	Metric	Database commit to first deny at the decision endpoint	p99 < 5 s
Is the cache carrying the load, or is the database?	Metric	Hit ratio on the read path	> 99%
Why was this specific request denied?	Log field	deciding_rule — which of the three permission planes answered	Present on 100% of decisions
Who acted on whose behalf?	Log field	sub and act on the decision line	Both present on every delegated call
Is a service dropping attribution?	Alert	Delegated calls reaching the log without act	Any occurrence → page
Is the database about to be stampeded?	Alert	Cache hit ratio	Below 95% for 10 minutes → warn

Every decision emits one structured line:

decision_id · sub · act · resource · action · decision · deciding_rule · source (cache|database) · latency_ms

S5: Failure Modes and Tradeoffs

Failure Modes

Failure	Blast radius	Detection	Mitigation	Residual risk
Source of truth unreachable on a cache miss	Users not already in the cache, across all firms	Query failure rate; deny rate on cache miss at the decision endpoint	Strict timeouts to free connections; single-flight so one miss does not become thousands of queries; fail closed	Uncached users are denied for the duration. By choice — the alternative is granting access we cannot verify
Write path (Auth-API) unavailable	No new logins, no token exchange, no permission change recorded. Existing sessions and all checks keep working, because the read path does not depend on it	Auth error rate; outbox backlog growth	Retry through the outbox; surface write failures to the caller instead of reporting success	A revocation the operator believes landed has not. No new access is granted meanwhile
Cache unavailable	Every check falls through to the database	Hit ratio below 95%	Single-flight on miss; decisions stay correct, only slower	Decision latency leaves its SLO while the cache is cold. Correctness is preserved; the p99 target is not
Invalidation never reaches the cache	Users whose permissions changed inside the window	Dead-letter queue growth; divergence count from the reconciliation job	Outbox, so the event is never lost separately from the write; on a failed cache write, delete the key rather than leave it stale; scheduled reconciliation	A stale privilege window until reconciliation runs. Revocation lag leaves its SLO silently — the system does not know it is stale
Firm federated IdP unreachable or certificate expired	Every user of that firm; no other firm affected	Auth failure rate per firm IdP	Per-firm circuit breaker with an explicit error; existing sessions continue until token expiry	No login for that firm until their IdP recovers. By choice — no local-credential fallback, that would be a bypass
OBO token used beyond the delegating user's scope	One downstream resource	Decision log entries where the act actor and the sub scope diverge	Downstream authorizes against sub, never the actor; narrowing at exchange time	Attribution depends on act surviving every hop; a service that drops it loses the audit trail

Tradeoffs

1. Chose to cache the full permission tree over caching resolved decisions. The cost is a coarser invalidation unit — one permission change invalidates a user's whole tree, not the single decision it affected — and a larger entry per user. I'll reconsider if the privilege sources are reduced from **3 to 2**.
2. Chose to revoke by *push (bus + outbox)* over *short TTL*. The cost is a new point of failure, maintenance of a new service, and eventual consistency. I'll reconsider if the **within seconds** revoke requirement is relaxed to **within minutes**.
3. Chose *Fail-Closed with Graceful Degradation* over *Fail-Open* on a cache miss with the database unreachable. The cost is two-sided: legitimate users outside the cache are denied, while users already cached keep stale privileges longer. I'll reconsider if the system moves from **external and internal users** to only **internal users**.
4. Chose the *Sync-API / Auth-API separation* over a *single Auth-API handling everything*. The cost is a new point of failure, added guardrails, and eventual consistency. I'll reconsider if the scale of permission checks drops from **tens of thousands** to just **thousands**.

Deviations

Two places where we depart from the specification, and why.

1. **Introspection (RFC 7662) is the spec's answer to "is this token still valid", and we do not call it.** At tens of thousands of checks per second an introspection round trip is the database round trip the brief forbids, relocated to a different host. We validate the token locally and re-evaluate the decision instead: revocation invalidates the decision, not the token.
2. **OIDC does not specify how a user reaches the right identity provider, so we decide it and accept the consequence.** Routing by email domain reveals whether an address is registered with a firm before authentication. We accept that leak rather than make every user name their firm, and we log the routing decision.